

The Biashara Mall Codebase — a guided tour

Who this is for: anyone who needs to understand how the code fits together before changing it — including us, in six weeks. `BUILD_LOG.md` says *what happened and why*, chronologically. This says *what exists now and how it works*. Current as of 2026-08-16 · **199 tests passing** · P0 complete · P1 complete · the buyer storefront is live.

1. The thirty-second version

A seller proves they own a TikTok account. We scrape their videos, an AI reads the product and price off each one, and the seller confirms. Buyers browse the result and message the seller on WhatsApp with the item and price already written for them.

The hard part is the price. Kenyan sellers don't put it in the caption — measured, 0 of 10 — so we read it from the cover image, and failing that, we *listen* to the video. That cascade is the heart of the system.

3,121 lines	application code	(29 Python files)
257 lines	templates	(3 Jinja2 files)
1,988 lines	tests	(199 tests)
599 lines	spikes	(throwaway probes against real APIs)
372 lines	migrations	(6, each reviewed before applying)

2. Running it

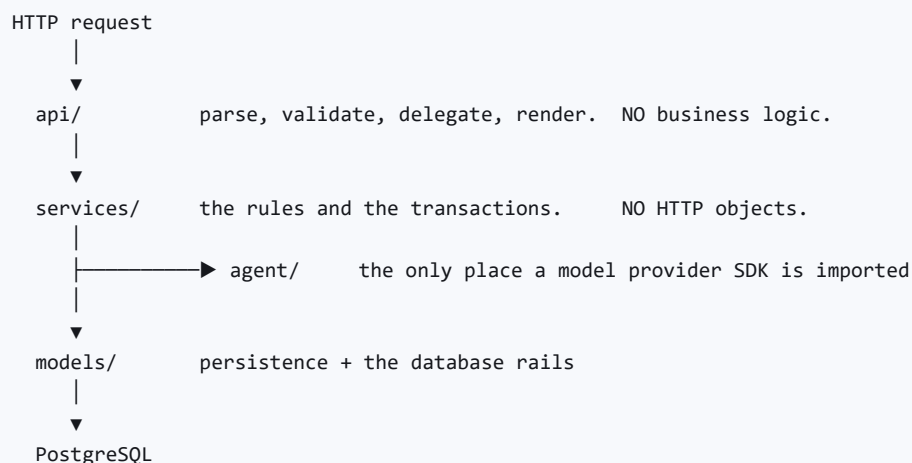
```
cd backend
.venv\Scripts\activate           # Windows

uvicorn app.main:app --reload     # http://localhost:8000
pytest                           # 199 tests
ruff check . && mypy app tests    # the quality gate
alembic upgrade head              # apply migrations

python scripts/seed_demo_shop.py  # a real, browsable demo shop
```

Two databases are required, and they must differ — see §8.

3. The layering rule



schemas/ sits sideways: Pydantic types used at both the HTTP boundary *and* as the contract handed to the LLM.
templates/ is rendered by api/ only.

A route containing business logic is in the wrong layer. A service building an HTTP response is in the wrong layer. That is the whole convention.

4. File by file

The spine

FILE	LINES	WHAT IT DOES
app/main.py	73	Builds the app, registers routers, mounts static + media. Owns nothing else.
app/config.py	142	Typed settings. The only file that reads the environment.
app/db.py	53	Engine, session factory, Base . Never build an engine elsewhere.
app/security.py	129	Argon2id hashing and JWT sessions. Nothing else touches a password.
app/dependencies.py	180	The session layer — cookies, current_account , the login wall.
app/templating.py	42	One Jinja environment and its filters, shared by every router.

Five details that are easy to undo by accident:

- **config.py normalises the database driver.** Providers hand out bare postgresql:// URLs; SQLAlchemy reads that as *psycopg2*, which we don't install. `_with_psycopg_driver()` rewrites it so `.env` holds the provider's URL verbatim on every environment.
- **config.py is lazy** (`@lru_cache`). An eager `settings = Settings()` would throw the moment anything imported the module, breaking tests and tooling.
- **db.py sets `pool_pre_ping=True`**. Cloud Postgres silently drops idle connections; without it the app hands a request a dead one and the user sees a 500 that vanishes on retry.
- **The storefront router is registered LAST in `main.py`, and must stay last.** It owns `/slug`, which would otherwise swallow `/health`, `/docs` and every future route. FastAPI matches in registration order, so anything

added below that line is unreachable. The other half of that guard is `RESERVED_SLUGS` in `services/accounts.py` — **a new top-level route needs a new entry there**, or a seller can claim the name and end up with an unreachable shop.

- `dependencies.py` **sets the session cookie's `Secure` flag from the environment**, not hardcoded. A `Secure` cookie is silently dropped over plain http, so hardcoding it on makes localhost log you in and immediately forget.

Models — where the rails live

FILE	LINES	WHAT IT HOLDS
<code>models/enums.py</code>	113	<code>Platform</code> , <code>IngestMethod</code> , <code>VerificationMethod</code> , <code>ProductStatus</code> , <code>ScrapeStatus</code> , <code>PriceSource</code>
<code>models/account.py</code>	63	A login. Email + Argon2id hash.
<code>models/seller.py</code>	115	A shop and its permanent public slug.
<code>models/social_account.py</code>	126	A connected platform account. Verified by construction.
<code>models/account_claim.py</code>	92	An <i>unproven</i> claim. Grants nothing.
<code>models/product.py</code>	279	A catalogue item. The largest and most constrained file.
<code>models/scrape_job.py</code>	97	One ingestion run, and the cache that keeps Apify affordable.

Why enums are strings + CHECK constraints, not native PG enums: adding a value to a native enum needs a migration and an exclusive lock. Adding one here is a code change. Platforms will grow.

Ingestion and AI

FILE	LINES	WHAT IT DOES
<code>schemas/tiktok.py</code>	121	The validation border. Apify JSON → validated objects.
<code>services/scraper.py</code>	208	Apify behind <i>our</i> interface. <code>ScraperEngine</code> is a Protocol.
<code>schemas/draft.py</code>	113	The contract the vision model must satisfy.
<code>agent/draft.py</code>	171	The provider seam. The only file that knows we use Gemini.
<code>services/drafting.py</code>	142	The cascade. Where the economics live.
<code>services/ingestion.py</code>	269	Scrape → cascade → <code>Product</code> rows. Where the money guards live.
<code>services/media.py</code>	94	Our own copies of covers, because theirs expire.

Accounts and the storefront

FILE	LINES	WHAT IT DOES
services/accounts.py	190	Signup, login, slug reservation. Anti-enumeration is the point.
services/verification.py	319	Proving a seller owns the account they claim.
services/storefront.py	126	What a buyer may see, and the WhatsApp handoff.
api/storefront.py	87	The two public routes. Thin: parse, delegate, render.
api/health.py	54	Liveness and readiness, deliberately separate.

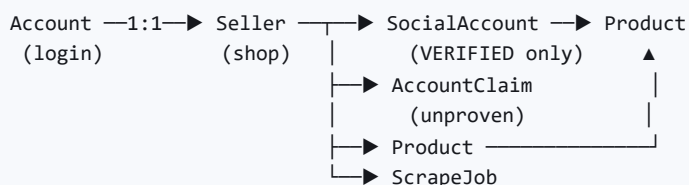
The web layer — the creator's workspace

FILE	LINES	WHAT IT DOES
api/auth.py	214	signup / login / logout. Renders the service's error, never its own.
api/dashboard.py	72	The shell, and the empty state that asks for a first account.
static/css/app.css	540	The design system. One <code>:root</code> block owns every colour.
templates/app_base.html	74	Workspace shell — topbar, nav, skip link.
templates/auth/	187	login.html , signup.html
templates/app/	85	dashboard.html
templates/base.html + storefront/	297	The buyer's storefront, on its own stylesheet

Two base templates, on purpose. `base.html` serves a stranger who arrives once from a WhatsApp chat on the cheapest data bundle available; `app_base.html` serves someone who signed up and comes back weekly. Keeping them apart means a change to the dashboard cannot slow down the storefront.

No Tailwind in the workspace. Its build step is one more thing to break in the deploy, and its CDN ships ~3MB of JS to generate CSS in the browser. `app.css` is 20.7KB uncompressed. Reversible: the components map onto utility classes, so adopting Tailwind later is a rewrite of one file.

5. The data model



Provenance is two dimensions, not one

COLUMN	ANSWERS	VALUES
<code>platform</code>	Where it came from	tiktok · instagram · facebook · jumia · manual
<code>ingest_method</code>	How it got here	profile_sync · single_link · upload

Separate because **only the second decides re-sync ownership**. A feed sync owns what it created and may update it. It must never touch a product the seller uploaded by hand — a seller who adds stock, syncs, and watches it vanish does not come back.

Verification: claims versus accounts

The two-table split is the whole design:

	ACCOUNTCLAIM	SOCIALACCOUNT
Means	"I say this is mine"	"This is proven mine"
Grants	nothing	everything
Lifetime	expires in 24h	permanent until disconnected
<code>verified_at</code>	—	NOT NULL

A row in `social_accounts` IS a verified account. There is no other kind. Nothing downstream filters for verified, because there is nothing to filter out. `_connect()` in `verification.py` is the only function that creates one, giving the invariant exactly one place to be got right.

The attack this defeats: a stranger types someone else's handle, we scrape her videos and photos, and they publish a storefront pointing at *their* WhatsApp number. Sales diversion, invisible to the buyer.

The rails — constraints in Postgres, not in service code

A model, a future agent, a migration script and a hand-written query all have to obey the database. Only application code has to *remember* to call a validator.

CONSTRAINT	PREVENTS
<code>published_requires_price</code>	the AI pushing an unpriced item live
<code>stock_non_negative</code>	overselling, at the storage layer
<code>price_positive</code>	KSh 0 — always a parse failure, never a giveaway
<code>price_plausible</code> ($\leq 10m$)	a phone number misread as a price reaching a storefront
<code>uq_products_platform_post</code>	a re-scrape duplicating instead of updating
<code>upload_has_no_post_id</code>	an uploaded product looking re-syncable
<code>manual_iff_upload</code>	an incoherent "tiktok upload"
<code>unit_quantity_and_label_together</code>	"KES 3,000 for 30" — of what?
<code>verified_at</code> NOT NULL	an unproven connection existing at all
<code>published_needs_whatsapp</code>	a live shop nobody can contact
<code>slug_format</code>	a URL a buyer cannot type
<code>failed_needs_error</code>	a scrape failure nobody can debug

Most model tests assert Postgres *refuses* something. A constraint nobody has watched fail is only a claim.

Money

`price_kes` is an **integer**, and it is the price of **one unit as sold** — which is not always one item.

```
price_kes      = 3000
unit_quantity  = 30
unit_label     = "pairs"
price_display  → "KES 3,000 for 30 pairs"
```

Found by spiking a real seller: @zumamitumbabales sells mitumba **bales**. Use `price_display`, never `price_kes` directly, in anything a buyer sees — including the WhatsApp message, or the seller and buyer start the conversation disagreeing about what was offered.

6. Three flows, traced

A. A seller proves they own an account

```
1. start_claim(seller, tiktok, "@zumamitumbabales")
   AccountClaim + code "soko-K7M2QP". Connects NOTHING.

2. Seller pastes the code into their TikTok bio.

3. check_claim(claim, scraper)
   expiry checked   ┌ both BEFORE the paid scrape
   attempts checked ┌ (capped at 10)
   |
   re-read bio → code present?
   |
   yes → _connect() → SocialAccount created, claim deleted
   no  → returns None. Still just a claim.
```

OAuth replaces steps 1–3 with one tap once TikTok and Meta approve our app. `complete_via_oauth()` is written and tested; the handle comes from the *provider*, never from anything the seller typed.

B. A verified account becomes draft products

```
1. sync_account(account, scraper, drafter)
   require_syncable()      disconnected? refuse
   cooldown check          synced < 24h ago? refuse (unless force)
   ScrapeJob created       RUNNING

2. scraper.fetch_profile()   capped at 30 recent posts
   failure → job marked FAILED with the reason, THEN raise
   (a failure the seller cannot see looks like "you have no videos")

3. For each post:
   already have it? → refresh METRICS ONLY, never the seller's edits
   new?             → cascade → store cover → Product (DRAFT)
   not a product?   → skipped, counted

4. account.last_synced_at set · job SUCCEEDED
```

Three money guards, each of which has cost someone a real bill:

1. **Once per day per account** — Apify bills per post.
2. **Skip posts already drafted** — that AI work is done and paid for.
3. **The video tier is opt-in** (`allow_video_tier=False` by default) — a bulk import that watched every clip would be ruinous.

One post failing never aborts the sync. The other twenty are still worth having; the failure lands in `warnings`.

C. A buyer reaches WhatsApp

```
GET /{slug}
  get_public_shop()      unknown OR unpublished → 404, identically
  get_public_products()  published only; in-stock first, then newest
                        sold-out items INCLUDED with a badge

GET /{slug}/{id}
  get_public_product()   scoped to this shop – a guessed id must not show
                        another seller's product under this header
  build_whatsapp_url()   wa.me link, message pre-written:

"Hi ZUMA MITUMBA BALES! I saw Mixed Ladies Sandals
(KES 3,000 for 30 pairs) on your Biashara Mall shop. Is it available?"
```

This handoff is the product. Everything upstream exists so this message can be written *for* the buyer instead of *by* them.

Sold-out items are shown on purpose: a buyer who saw the item on TikTok and finds nothing assumes the shop is dead. A "Sold out" badge says it is real and worth asking about — and gives Soko Intel a restock signal later.

7. Two lessons the code carries

Cover URLs expire. TikTok signs them with an `x-expires` timestamp. Spike 02 flagged it; the first demo shop then *proved* it by rendering ten broken images a week after the scrape. `services/media.py` stores our own copy, and `Product` holds a **relative** path — so moving to object storage later changes one constant and nothing else.

`scripts/seed_demo_shop.py` **scrapes live rather than replaying saved data**, for the same reason. Its first version replayed spike 01's payload and produced exactly that wall of broken images.

8. Tests

FILE	TESTS	PROTECTS
<code>test_auth.py</code>	43	Hashing, sessions, slugs, anti-enumeration
<code>test_models.py</code>	42	The database rails. Mostly asserts Postgres refuses .
<code>test_verification.py</code>	35	That an unverified account is unrepresentable
<code>test_scraper.py</code>	26	The validation border, against the real captured payload
<code>test_ingestion.py</code>	19	The money guards and the re-sync rail
<code>test_drafting.py</code>	16	The cascade — which tiers ran , i.e. cost
<code>test_config.py</code>	14	Settings, hermetic (<code>_env_file=None</code>)
<code>test_health.py</code>	4	Liveness vs readiness

Conventions worth knowing:

- **Externals are always mocked.** No test calls Apify, Gemini, Meta or Daraja.

- **Scraper fixtures come from the real payload** captured by spike 01. A test passing against a payload we imagined proves nothing.
- **Cascade and ingestion tests assert cost**, not only output — which tiers ran, whether a scrape happened at all. A regression that quietly escalated every item would pass a naive correctness test and arrive as a bill.
- `tests/factories.py` **cannot build an unverified account**. The thing does not exist. That is the guarantee proving itself.

Two databases, on purpose

`TEST_DATABASE_URL` must be set and must **differ** from `DATABASE_URL`.

- Unset → database-backed tests **skip** (a visible gap).
- Identical → the suite **refuses to run**.

The `_schema` fixture **drops and recreates**, because `create_all` only adds missing tables and never alters existing ones — that drift once produced 25 failures reading *"column unit_quantity does not exist"*.

9. Spikes

`spikes/` holds throwaway probes against real APIs. Nothing in `app/` imports them; they are kept because their **findings** justify the design.

SPIKE	COST	FOUND
01 · Apify profile	paid	Payload shape. 0/10 captions have prices. Bio carries phone numbers.
02 · Offline analysis	free	No transcriptions. Cover URLs expire. Clips are 9–24s.
03 · Gemini cover	paid	0/4 — and the model correctly refused to invent a price.
04 · Gemini video	paid	3/3 . And <code>price_evidence</code> revealed bulk pricing.

Spike 01 writes its raw response to `spikes/out/`, so every later question is answered offline for free. **Spend once, analyse many times.**

10. Where to start reading

In this order — each builds on the last:

1. `app/models/product.py` — the domain, and every rail. Start here.
2. `app/schemas/draft.py` — what we demand of the AI, and what we deliberately *don't* let it decide.
3. `app/services/drafting.py` — the cascade. The clearest expression of how cost shapes this system.
4. `app/services/ingestion.py` — where the cascade meets the database, and where the money guards live.
5. `app/services/verification.py` — read the module docstring; the claim/account split is the least obvious decision in the codebase.
6. `app/services/accounts.py` — read `authenticate()` closely; the anti-enumeration reasoning is non-obvious.
7. `tests/test_models.py` — the rails, proven. Reads as a specification.

Every file opens with a header docstring saying what it does, the pipeline it sits in, and **why it is shaped that way**. If something looks strange, that docstring almost certainly explains it — and if it doesn't, that is a documentation bug worth fixing.

11. What exists, and what doesn't

	STATUS
Foundation, config, DB, health, CI	✓
Models + the database rails	✓
Scraper adapter + validation border	✓
Draft agent + the price cascade	✓
Ownership verification (bio-code)	✓
Ingestion — sync, guards, re-sync rail	✓
Media storage	✓
Buyer storefront — shop + product + WhatsApp handoff	✓
Design system	✓
Session layer — cookies, login wall, safe redirects	✓
Signup / login / logout pages	✓
Dashboard shell + empty state	✓
Connect-an-account screens — claim, show the code, check it	✗ next
<code>Post</code> model + metric snapshots (the time series)	✗
Analytics sync + analytics page	✗
Review queue + publish UI	✗
Manual upload path	✗
Single-link ingestion path	✗ scraper supports it; nothing calls it
Deploy to Railway	✗ no start command committed yet
WhatsApp channel	🕒 gated on Meta verification
Orders + M-Pesa	✗ P4
Soko AI, Soko Intel	✗ P5–P8

The gap that matters: a creator can now sign up, sign in and reach a dashboard in a browser — but the dashboard's one button goes nowhere. Connecting an account still requires Python, so nothing real appears on that page yet. The storefront works and is browsable; getting products *into* it currently requires `scripts/seed_demo_shop.py`.

Two things are being thrown away right now, and only one is recoverable.

`services/ingestion.py` discards every post the model judges not a product — correct for a catalogue, wrong for analytics, where a talking-head video still has views a creator is paying to understand. And `Product.views` is *overwritten* on each sync, so nothing can answer "am I growing?".

The first is a filter to relax. The second is history that cannot be backfilled — every day without `PostMetricSnapshot` is a day of data gone permanently. That is why Step 5 of `PHASE_0.md` is a migration with no UI attached.

Connect-an-account is the next thing built.